Machine Coding: Persistent Counter (localStorage)

- **Problem:** Build a counter with increment, decrement and reset, plus a selectable step (1 / 5 / 10), limited to 0–1000. The count and step must survive a page reload (localStorage), stay in sync across browser tabs, and never crash or show `NaN` when storage is empty, corrupted, blocked or full.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components, hooks and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Range, step, sync rules decided
Use cases + architecture + state	5 min	Table, `useLocalStorage` contract
Counter with `useState` (+, −, reset)	8 min	Working counter
Persist: lazy read + write effect	10 min	Survives reload
Safe parsing, bounds, step, storage failures	12 min	Never `NaN`, never crashes
Cross-tab sync + a11y + self-check	10 min	Robust + proven
Manual test (reload, 2 tabs, corrupt value) + wrap-up	10 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"Anything I read from localStorage is untrusted input: it might be missing, a string, `NaN`, or out of range. So I*

parse and validate it once when I read it, and fall back to a default." Then demo by typing `localStorage.setItem("counter.value", "abc")` in the console and reloading.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show the count; buttons + step and − step; Reset to 0.
- Step selector: 1, 5 or 10.
- Range 0–1000: buttons disable at the limits; a step that would cross a limit stops at the limit.
- Count and step are restored after a reload.
- Changing the count in one tab updates the other open tabs.
- Show a warning if the browser won't let us save.

Non-functional requirements

Concern	What it means here
Performance	One synchronous `localStorage` read on mount (lazy initialiser), one small write per change.
Scalability	`useLocalStorage(key, initial, parse)` is generic: any persisted setting (theme, filters, draft) is one line.
Error handling	Missing key, invalid JSON, wrong type, out of range, blocked storage (Safari private mode, disabled cookies), quota exceeded: all fall back safely.
Accessibility	`<output aria-live="polite">` announces the count; buttons have clear labels ("Increase by 5"); the step uses a radio group in a `fieldset`; visible focus; disabled states.
Reusability	Pure `readStorage` / `writeStorage` (testable with a fake storage) + the hook.
Extensibility	Undo history, keyboard shortcuts, multiple counters or server sync build on the same hook.

Clarifying questions to ask the interviewer

1. Can the count go negative? Is there a maximum?
2. Should the step be configurable, and should it also be persisted?
3. What should Reset do: set 0 (and keep saving), or remove the key from storage?
4. Should two open tabs stay in sync?
5. localStorage or sessionStorage (per tab, cleared on close)? Or a server?
6. What should happen if storage is unavailable: silently work in memory, or warn the user?

Assumptions we will build with

- Range 0–1000; whole numbers; a step that would overshoot stops at the limit (clamp).
- Steps 1 / 5 / 10, persisted too.
- Reset sets 0 (still saved), so a reload shows 0.
- Keys `counter.value` and `counter.step`, values stored as JSON.
- Tabs sync through the `storage` event; if storage fails, the app works in memory and shows a warning.

2. Identify interactions / use cases

Action / input	Expected behavior / output
First visit (nothing stored)	Count 0, step 1; − disabled
Click + with step 1 / 5 / 10	Count increases by the step; saved
Click − at 3 with step 5	Count becomes 0 (clamped), not −2
Count at 1000	+ disabled
Choose step 5	"+ 5" / "− 5" labels; step saved
Click Reset	Count 0; Reset disabled at 0
Reload the page	Same count and step
Click fast many times	Every click counts (functional updates)
Stored value is `"abc"` / `"7"` / `null` / `3.9` / `99999`	Falls back to 0 / 0 / 0 / 3 / 1000; never `NaN`
Stored step is `7`	Falls back to step 1
Change the count in tab A	Tab B shows the new count without reloading
Clear storage in another tab (`localStorage.clear()`)	This tab goes back to the defaults
Storage blocked or full	The counter still works; warning "changes won't be kept"
Keyboard	Tab to the buttons, Enter/Space work; arrow keys switch the step

3. Design the architecture

Module	Responsibility
`App`	Uses two persisted states (count, step); handlers; renders the UI
`hooks/useLocalStorage.js`	`useState` + lazy read + write effect + cross-tab `storage` listener + save-failure flag
`utils/storage.js`	`readStorage(key, fallback, parse)` / `writeStorage(key, value)` wrapped in try/catch
`utils/counter.js`	`MIN` , `MAX` , `STEPS` , `clamp` , `parseCount` , `parseStep` (validation rules)

```
App
 ├── [count, setCount, countSaveFailed] = useLocalStorage("counter.value", 0, parseCount)
 ├── [step,  setStep,  stepSaveFailed]  = useLocalStorage("counter.step", 1, parseStep)
 ├── <output aria-live> count
 ├── − step / + step buttons (clamped, disabled at bounds)
 ├── step radio group (1 / 5 / 10)
 ├── Reset
```

```
  └── warning if saving failed
useLocalStorage → utils/storage.js → window.localStorage
                → window "storage" event (other tabs)
```

Start simple. First version: `useState(0)` + three buttons. Then persist it inline in `App`, then extract `useLocalStorage` once a second value (step) needs the same logic.

4. Design the state / data model

State	Where	Purpose
`count`	useLocalStorage("counter.value")	The counter value (0–1000)
`step`	useLocalStorage("counter.step")	Increment size (1, 5, 10)
`saveFailed`	inside each useLocalStorage	Whether the last write failed

```
// utils/counter.js
export const MIN = 0, MAX = 1000, STEPS = [1, 5, 10];
export const clamp = (n) => Math.min(MAX, Math.max(MIN, n));
export const parseCount = (v) => (Number.isFinite(v) ? clamp(Math.trunc(v)) : undefined);
export const parseStep = (v) => (STEPS.includes(v) ? v : undefined);

// App
const [count, setCount, countSaveFailed] = useLocalStorage("counter.value", 0, parseCount);
const [step, setStep] = useLocalStorage("counter.step", 1, parseStep);
const increment = () => setCount((c) => clamp(c + step));
```

Senior note

- **React state is the source of truth; localStorage is a mirror.** We read storage once (lazy initialiser), then write it after every change. We never read it during render.
- **Derived, not stored:** the disabled states (`count <= MIN` , `count >= MAX` , `count === 0`), the button labels and the warning visibility.
- **Validate at the boundary:** `parseCount` / `parseStep` run when reading storage (untrusted). Inside the app the value is always a valid integer.
- **Parse functions are defined outside the component**, so their identity is stable and the `storage` listener effect doesn't re-subscribe on every render.

5. Design the data structures and contracts

What's stored (JSON-encoded, so types survive: a number stays a number):

```
counter.value → "15"
counter.step  → "5"
```

Contracts

```
readStorage(key, fallback, parse = v => v, storage = localStorage) → value | fallback   // never throws
writeStorage(key, value, storage = localStorage) → true | false                          // never throws
useLocalStorage(key, initialValue, parse) → [value, setValue, saveFailed]
// parse(decoded) returns the validated value, or undefined to use the fallback
```

Storage event (from other tabs):

```
{ key: "counter.value", oldValue: "15", newValue: "20", storageArea: localStorage }
// key === null after localStorage.clear(); the event never fires in the tab that made the change
```

6. Trace the data flow

User clicks "+ 5" in tab A while tab B is open:

```
Tab A: click "+ 5"
  ↓ setCount(c => clamp(c + 5)) → 15 → 20
  ↓ React re-renders: <output> shows 20 (aria-live announces it); + / − disabled states recomputed
  ↓ useLocalStorage effect [key, value]: writeStorage("counter.value", 20) → localStorage = "20"
  ↓ browser fires a "storage" event in every OTHER tab of the same origin
Tab B: onStorage(e) → e.key === "counter.value" → readStorage(...) → parseCount(20) → setValue(20)
  ↓ Tab B re-renders with 20; its write effect sets the same "20" → no new event (no ping-pong)
```

7. Important technical challenges

Challenge 1: reading storage safely (never `NaN`)

- *Problem:* `Number(localStorage.getItem("count"))` turns `"abc"` into `NaN`, and the counter shows "NaN" forever (NaN + 1 is NaN). A user or extension can put anything there, an old app version may have stored a different shape, and `getItem` itself **throws** when storage is blocked.
- *Solution:* JSON-decode inside try/catch, validate with a `parse` function, fall back to the default.

```
export function readStorage(key, fallback, parse = (v) => v, storage = globalThis.localStorage) {
  try {
    const raw = storage.getItem(key);
    if (raw === null) return fallback;
    const value = parse(JSON.parse(raw));
    return value === undefined ? fallback : value;
  } catch {
    return fallback;
  }
}
export const parseCount = (v) => (Number.isFinite(v) ? clamp(Math.trunc(v)) : undefined);
```

Challenge 2: when to read and when to write

- *Problem:* `useState(Number(localStorage.getItem(...)))` reads storage on **every render** (the argument is evaluated each time). The first version's Clear button did `removeItem` and then `setCount(0)`, but the effect immediately writes `"0"` back, so the `removeItem` was pointless.
- *Solution:* A lazy initialiser reads once; one effect writes after every change; Reset just sets 0.

```
const [value, setValue] = useState(() => readStorage(key, initialValue, parse)); // runs once
useEffect(() => { setSaveFailed(!writeStorage(key, value)); }, [key, value]);
const reset = () => setCount(0);
```

Challenge 3: syncing across tabs

- *Problem:* Two tabs each keep their own React state. Tab A goes to 20, tab B still shows 15, and the next click in B saves 16, silently overwriting A's work.
- *Solution:* Listen to the `storage` event (it fires in *other* tabs when a key changes), re-read and validate. Writing the same value again fires no event, so there's no infinite loop.

```
useEffect(() => {
  const onStorage = (e) => {
    if (e.storageArea !== localStorage) return;
    if (e.key !== null && e.key !== key) return;            // null = localStorage.clear()
    setValue(e.newValue === null ? initialValue : readStorage(key, initialValue, parse));
  };
  window.addEventListener("storage", onStorage);
  return () => window.removeEventListener("storage", onStorage);
}, [key, initialValue, parse]);
```

Challenge 4: correct updates and bounds

- *Problem:* `setCount(count + step)` twice in the same event uses the same stale `count`, so only one increment lands. Decrementing 3 by 5 gives −2, which breaks the "no negatives" rule.
- *Solution:* Functional updates + `clamp`, and disabled buttons at the limits.

```
const decrement = () => setCount((c) => clamp(c - step));
<button onClick={decrement} disabled={count <= MIN} aria-label={`Decrease by ${step}`}>− {step}</button>
```

8. Implement in the correct order

#	Step	Working result
1	`useState(0)` + / − / Reset buttons	Working counter
2	Functional updates `setCount(c => c + 1)`	No lost clicks
3	Persist: lazy read + write effect inline in App	Survives reload
4	`JSON.parse` + try/catch + `parseCount` (finite, integer)	No `NaN`, no crash
5	Bounds 0–1000 with `clamp` + disabled buttons	Valid range

#	Step	Working result
6	Extract `useLocalStorage(key, initial, parse)`	Reusable
7	Step 1/5/10 persisted with a second hook call	Configurable step
8	`storage` event listener (incl. `clear()`)	Tabs stay in sync
9	`writeStorage` returns false → warning banner	Graceful when blocked/full
10	`<output aria-live>`, labels, radio group, focus styles	Accessible
11	Node self-check with a fake / blocked storage	Logic proven

9. Cover all states and edge cases

State / edge case	Handling
First visit	Defaults (0, step 1)
Normal use	Every change saved
Reload	Values restored once, before the first paint
Key missing	Default
Invalid JSON (`abc`)	Default
Wrong type (`"7"` string, `true`, `{}`)	Default
Decimal (`3.9`)	Truncated to 3
Out of range (`-3`, `99999`)	Clamped to 0 / 1000
Invalid step (`7`)	Step 1
At 0 / at 1000	− / + disabled
Step overshoots a bound	Clamped to the bound
Fast clicking	Functional updates, no lost clicks
Another tab changes the value	Synced through the `storage` event
Another tab clears storage	Back to defaults
Storage blocked (private mode / policy)	Reads fall back; writes fail → warning; app keeps working in memory
Quota exceeded	Same as blocked
Server-side rendering (no `window`)	Not applicable here; would need a `typeof window` guard (see Q19)

10. Final structure

```
_08_persistant_counter_app_localstorage/
├── index.html
└── src/
    ├── main.jsx
```

```
    ├── index.css                ← global reset
    ├── App.jsx                  ← counter UI using two persisted states
    ├── App.css
    ├── hooks/
    │   └── useLocalStorage.js   ← persisted useState + cross-tab sync
    └── utils/
        ├── storage.js           ← readStorage / writeStorage (never throw)
        └── counter.js           ← MIN, MAX, STEPS, clamp, parseCount, parseStep
```

11. Full working code

src/utils/storage.js

```
// Safe localStorage access: storage can be missing, blocked (private mode), full, or hold garbage.

// `parse` validates the decoded value and returns undefined when it is not acceptable.
export function readStorage(key, fallback, parse = (v) => v, storage = globalThis.localStorage) {
  try {
    const raw = storage.getItem(key);
    if (raw === null) return fallback;
    const value = parse(JSON.parse(raw));
    return value === undefined ? fallback : value;
  } catch {
    return fallback; // blocked storage or invalid JSON
  }
}

// Returns false when the value could not be saved (quota exceeded, storage disabled).
export function writeStorage(key, value, storage = globalThis.localStorage) {
  try {
    storage.setItem(key, JSON.stringify(value));
    return true;
  } catch {
    return false;
  }
}
```

src/utils/counter.js

```
export const MIN = 0;
export const MAX = 1000;
export const STEPS = [1, 5, 10];

export const clamp = (n) => Math.min(MAX, Math.max(MIN, n));

// Validators for values read from storage: return undefined to fall back to the default.
```

```
export const parseCount = (v) => (Number.isFinite(v) ? clamp(Math.trunc(v)) : undefined);
export const parseStep = (v) => (STEPS.includes(v) ? v : undefined);
```

src/hooks/useLocalStorage.js

```
import { useEffect, useState } from "react";
import { readStorage, writeStorage } from "../utils/storage";

// useState that survives reloads and stays in sync across tabs.
// `parse` must be a stable function (defined outside the component).
export function useLocalStorage(key, initialValue, parse) {
  const [value, setValue] = useState(() => readStorage(key, initialValue, parse)); // read once, lazily
  const [saveFailed, setSaveFailed] = useState(false);

  // Write after every change. Writing the same value again fires no "storage" event, so tabs don't
ping-pong.
  useEffect(() => {
    setSaveFailed(!writeStorage(key, value));
  }, [key, value]);

  // Another tab changed (or cleared) this key -> take its value.
  useEffect(() => {
    const onStorage = (e) => {
      if (e.storageArea !== localStorage) return;
      if (e.key !== null && e.key !== key) return; // e.key is null after localStorage.clear()
      setValue(e.newValue === null ? initialValue : readStorage(key, initialValue, parse));
    };
    window.addEventListener("storage", onStorage);
    return () => window.removeEventListener("storage", onStorage);
  }, [key, initialValue, parse]);

  return [value, setValue, saveFailed];
}
```

src/App.jsx

```
import "./App.css";
import { useLocalStorage } from "./hooks/useLocalStorage";
import { MAX, MIN, STEPS, clamp, parseCount, parseStep } from "./utils/counter";

function App() {
  const [count, setCount, countSaveFailed] = useLocalStorage("counter.value", 0, parseCount);
  const [step, setStep, stepSaveFailed] = useLocalStorage("counter.step", 1, parseStep);

  // Functional updates: correct even when clicks are batched or come fast.
  const increment = () => setCount((c) => clamp(c + step));
```

```
  const decrement = () => setCount((c) => clamp(c - step));
  const reset = () => setCount(0);

  return (
    <main className="container">
      <h1>Persistent Counter</h1>

      <output className="count" aria-live="polite" aria-label={`Count: ${count}`}>
        {count}
      </output>

      <div className="actions">
        <button type="button" className="minus" onClick={decrement} disabled={count <= MIN} aria-label=
{`Decrease by ${step}`}>
          − {step}
        </button>
        <button type="button" className="plus" onClick={increment} disabled={count >= MAX} aria-label=
{`Increase by ${step}`}>
          + {step}
        </button>
      </div>

      <fieldset className="steps">
        <legend>Step</legend>
        {STEPS.map((s) => (
          <label key={s} className={s === step ? "active" : ""}>
            <input type="radio" name="step" value={s} checked={s === step} onChange={() => setStep(s)}
/>
            {s}
          </label>
        ))}
      </fieldset>

      <button type="button" className="reset" onClick={reset} disabled={count === 0}>
        Reset
      </button>

      <p className="hint">
        Range {MIN}–{MAX}. Saved in this browser; open another tab to see it sync.
      </p>
      {(countSaveFailed || stepSaveFailed) && (
        <p role="alert" className="warning">
          Storage is unavailable, so changes won't be kept after a reload.
        </p>
      )}
    </main>
  );
```

```
}

export default App;
```

src/App.css

```
body { background: #f3f4f6; font-family: system-ui, sans-serif; color: #111827; }
.container { max-width: 420px; margin: 60px auto; padding: 28px 20px; background: white; border-radius:
12px; text-align: center; display: flex; flex-direction: column; align-items: center; gap: 18px; }
h1 { font-size: 24px; }

.count { font-size: 64px; font-weight: 700; font-variant-numeric: tabular-nums; min-width: 3ch; }

.actions { display: flex; gap: 12px; }
button { padding: 10px 20px; border: none; border-radius: 8px; font-size: 18px; color: white; cursor:
pointer; }
button:disabled { opacity: 0.4; cursor: not-allowed; }
button:focus-visible { outline: 3px solid #a5b4fc; outline-offset: 2px; }
.plus { background: #16a34a; }
.minus { background: #dc2626; }
.reset { background: #4b5563; font-size: 15px; }

.steps { display: flex; gap: 8px; border: none; align-items: center; }
.steps legend { float: left; margin-right: 8px; font-weight: 600; }
.steps label { padding: 6px 14px; border: 1px solid #d1d5db; border-radius: 999px; cursor: pointer; }
.steps label.active { background: #4f46e5; border-color: #4f46e5; color: white; }
.steps label:has(input:focus-visible) { outline: 3px solid #a5b4fc; }
.steps input { position: absolute; opacity: 0; width: 0; height: 0; }

.hint { font-size: 13px; color: #6b7280; }
.warning { font-size: 14px; color: #b45309; background: #fef3c7; padding: 8px 12px; border-radius: 6px;
}
```

src/index.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
```

```
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled: missing / invalid / wrong-type / out-of-range stored values, blocked or full storage (with a warning), stale-state updates, bounds and overshooting steps, sync between tabs (including `clear()`), no ping-pong writes, accessible announcements and controls.

Trade-offs to mention to the interviewer

- *localStorage vs sessionStorage vs IndexedDB vs server:* localStorage is simple, synchronous, ~5 MB per origin, and shared by all tabs. sessionStorage is per tab. IndexedDB is async and large. A server is needed for cross-device or multi-user data.
- *Write in an effect vs inside the setter:* the effect keeps one write path, but runs after render (a tiny delay). Writing inside a custom setter is immediate, but you must handle functional updates yourself.
- *Clamp vs disable when a step overshoots:* clamping lets you reach exactly 0/1000 with big steps; disabling is stricter. Ask which the product wants.
- *JSON encoding:* types survive (a number stays a number), at the cost of `JSON.parse` + a try/catch.
- *Two keys vs one object:* two keys are simpler to validate separately; one object (`{ value, step, version }`) is easier to migrate.

What to add with more time

- Unit tests for the hook (`renderHook` , a mocked `localStorage` , dispatching a `StorageEvent` to test sync).
- Keyboard shortcuts (↑ / ↓ / R), long-press repeat on the buttons.
- Undo / history of changes.
- A schema version in the stored data, with migrations.
- `useSyncExternalStore` -based store for multiple components sharing the same key in one tab.
- BroadcastChannel for richer cross-tab messages.

13. Counter questions the interviewer may ask

Requirements and design

Q1. localStorage vs sessionStorage vs cookies: when do you use which?

localStorage: persists until cleared, shared by all tabs of an origin, ~~5 MB, never sent to the server. sessionStorage: per tab, gone when the tab closes. Cookies: small~~ (4 KB), sent with every request, needed for server auth (`HttpOnly`). A counter preference fits localStorage.

Q2. Is localStorage synchronous? Why does it matter?

Yes. Every `getItem` / `setItem` blocks the main thread. Small values are fine; large or frequent writes (e.g. on every keystroke of a big document) can cause jank. Then debounce the writes or use IndexedDB (async).

Q3. What can you NOT safely store in localStorage?

Secrets and auth tokens: any script on the page (XSS, a compromised third-party script) can read it. Use `HttpOnly` cookies for sessions. Also nothing that must stay private on shared computers.

State design

Q4. Why `useState(() => read())` instead of `useState(read())`?

`useState(read())` calls `read()` on every render (the result is just ignored after the first). The function form (lazy initialiser) runs only once, on mount.

Q5. Why write in a `useEffect` instead of inside each click handler?

One write path for every way the value changes (clicks, reset, sync from another tab), so you can't forget one. Handlers stay pure state updates.

Q6. Why `setCount(c => c + step)` instead of `setCount(count + step)`?

`count` in the closure is the value from the last render. Two updates in the same event (or quick batched clicks) would both use the old value, and one would be lost. The functional form always gets the latest state.

Q7. The first version used `React.memo` on the button and `useCallback` on the handlers. Was that useful?

Barely: the button is cheap to render, and memoisation has its own cost and complexity. It only helps when a child is expensive or long lists re-render. Here I removed it and kept the code simple; mention it as a measurement-driven optimisation.

Hard parts

Q8. What does `Number(localStorage.getItem("count"))` return in each case?

Missing key: `Number(null)` = 0 (by luck). `"abc"` → `NaN`. `""` → 0. `"12px"` → `NaN`. `"1e3"` → 1000. That's why we decode with `JSON.parse` and validate with `Number.isFinite` + range checks instead of trusting `Number()`.

Q9. When does the `storage` event fire?

In **other** tabs/windows of the same origin, when localStorage changes. Not in the tab that made the change, and not when `setItem` writes the same value. `e.key` is `null` after `localStorage.clear()`.

Q10. Could the cross-tab sync cause an infinite loop?

No. Tab B receives 20 and sets its state to 20; its effect writes `"20"`, which is the same as the stored value, so no new event fires. If values were transformed differently in each tab (e.g. different clamping), it could ping-pong, so validate the same way everywhere.

Q11. How do you handle storage being unavailable?

Safari private mode (older versions), disabled cookies, enterprise policies or sandboxed iframes can make `localStorage` throw on access, and a full quota throws on `setItem`. Wrap reads and writes in try/catch, fall back to in-memory state, and tell the user it won't be saved.

Q12. How would you change the stored format later without breaking users?

Store a version: `{ version: 2, value: 15, step: 5 }`. On read, migrate old versions (v1 was a bare number) or fall back to defaults. Never assume the stored shape matches the current code.

Implementation steps

Q13. How do you test a hook that uses localStorage?

`renderHook(() => useLocalStorage("k", 0, parse))` with Testing Library. Pre-fill `localStorage` (jsdom has one) to test reading; call the setter and assert `localStorage.getItem`; dispatch `new StorageEvent("storage", { key: "k", newValue: "5", storageArea: localStorage })` to test sync. The pure `readStorage` / `writeStorage` take the storage as a parameter, so they can be tested with a fake or throwing storage.

Q14. How would you add keyboard shortcuts (↑ / ↓)?

A `keydown` listener on `window` in an effect (with cleanup), ignoring events from inputs/textareas, calling the same `increment` / `decrement`. Document the shortcuts for users (e.g. `aria-keyshortcuts`).

Q15. How do you announce the count to screen readers?

`<output aria-live="polite">` makes screen readers read the new value after each change without moving focus. Buttons get explicit labels ("Increase by 5"), because "+ 5" alone is read as "plus five" without context.

Performance and scale

Q16. The user holds the + button and it fires 30 times per second. Is writing to localStorage each time a problem?

Small values: not really, but it's wasted work. You can debounce/throttle the write (e.g. 200 ms) while keeping the UI state instant, and flush on `pagehide` / `visibilitychange` so the last value isn't lost.

Q17. Many components on the same page use the same key. How do you keep them in sync within one tab?

The `storage` event doesn't fire in the same tab. Use a small external store (a module-level value + subscribers) read with `useSyncExternalStore`, or React Context, so all components share one state and one write.

Q18. How would you sync the counter across devices?

Save to a backend (debounced) with the user id; load it on start; resolve conflicts (last-write-wins with timestamps, or server-authoritative increments). Use WebSockets/SSE for live updates across devices.

Trade-offs and "why not X?"

Q19. What breaks with server-side rendering (Next.js)?

`localStorage` and `window` don't exist on the server, so reading in the initializer crashes or causes a hydration mismatch (the server renders 0, the client 15). Fix: render the default on the server, then read storage in an effect after mount (or `useSyncExternalStore` with a `getServerSnapshot`).

Q20. Why not BroadcastChannel instead of the `storage` event?

`BroadcastChannel` is great for sending any message between tabs (not just storage changes) and doesn't need storage at all. Here the data already lives in localStorage, and the `storage` event gives us sync for free; I'd use BroadcastChannel for richer cross-tab coordination (e.g. "logout everywhere").

Q21. Why not just use a library like `use-local-storage-state` or Zustand `persist`?

In production, those handle SSR, serialisation, sync and edge cases well. In an interview, writing `useLocalStorage` shows you understand lazy initialisation, effects, validation of untrusted input and the `storage` event.